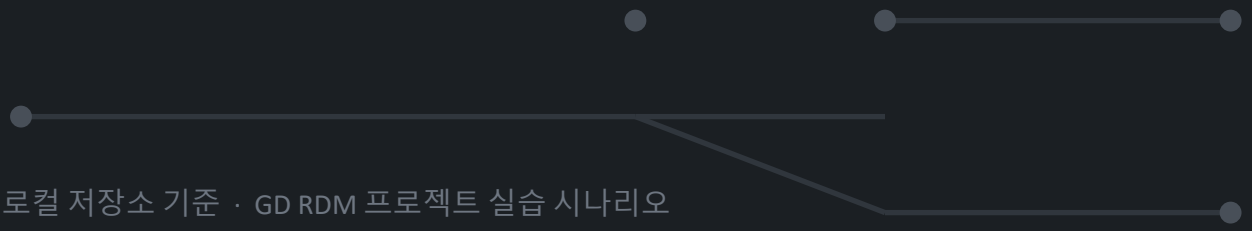
GIT 교육 자료 · 실습형 강의

Git 완전정복

하나의 시나리오로 배우는 모든 핵심 명령어

가상의 프로젝트를 처음부터 만들어가며 **실제 실행 결과**와 함께
Git의 모든 핵심 명령어를 순서대로 익힙니다.



로컬 저장소 기준 · GD RDM 프로젝트 실습 시나리오

OVERVIEW

오늘 배우는 5개 챕터

01

저장소 시작

init · add · commit · diff

02

브랜치와 병합

branch · checkout · merge · 충돌 해결

03

되돌리기

restore · amend · reset(soft/mixed/hard)

04

안전한 되돌리기 & 임시보관

revert · stash

05

이력 탐색

tag · show · blame · log graph

핵심 개념

시작하기 전에 — Git의 3가지 영역



git restore · git reset → 되돌리는 명령어들은 반대 방향으로 흐릅니다

01

저장소 시작



init · add · commit · diff — 파일을 추적하고 기록을 남기는 기본 흐름

git init — 모든 것의 출발점



`git init`은 현재 폴더에 숨겨진 **.git** 디렉터리를 만듭니다.
이 폴더 하나에 모든 커밋·브랜치·이력이 저장됩니다.

```
$ git init
Initialized empty Git repository in .../demo/.git/

$ git status
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```



git status 는 가장 먼저, 가장 자주 쳐야 하는 명령어입니다. 지금 Working Directory와 Staging Area가 어떤 상태인지 항상 알려줍니다.

git add → git commit — 파일 추적과 기록

Untracked

README.md 생성 직후

↓

Staged

git add README.md

↓

Committed

git commit -m "..."

```
$ git add README.md
$ git status
Changes to be committed:
  new file:   README.md

$ git commit -m "초기 커밋: README 추가"
[main (root-commit) 7d8eece] 초기 커밋: README 추가
 1 file changed, 2 insertions(+)
```

7d8eece 는 이 커밋의 고유 해시(ID)입니다. 모든 커밋은 이런 SHA-1 해시로 식별됩니다 (40자리, 보통 7자리로 축약).

```
$ git log
commit 7d8eece64b5f5a080c4faf6502aab7eb084c3ba6
Author: 최상길 <sangkil@example.com>
Date:   Fri Jun 19 09:29:03 2026 +0000    →    초기 커밋: README 추가
```

git diff — 무엇이 바뀌었는지 정확히 보기

```
$ git diff
--- a/src/rdm_data_generator.py
+++ b/src/rdm_data_generator.py
-    pass
+    return pd.DataFrame({"col1": range(rows)})
```

git diff

아직 add 안 한 변경사항

Working Directory ↔ Staging Area

git diff --cached

add는 했지만 commit 안 한 변경사항

Staging Area ↔ 마지막 커밋

`git add -A`는 `git add .`과 비슷하지만, 삭제된 파일까지 포함해 모든 변경을 스테이징합니다.

.gitignore — 추적하지 말아야 할 파일 제외하기

보안 환경 팁 — 실제 데이터(real data)가 외부로 나가면 안 되는 프로젝트에서 .gitignore는 특히 중요합니다.

```
# .gitignore
__pycache__/
*.pyc
.venv/
data/real/    ← 실제 데이터 폴더
```

```
$ git status
Untracked files:
  .gitignore
  src/
↑ secret.csv 안 보임!
```

핵심: **data/real/**을 등록하면 Git이 그 안의 어떤 파일도 추적 후보로조차 보여주지 않습니다.
git add .처럼 한 번에 모든 파일을 추가하더라도 민감한 데이터가 실수로 커밋되는 사고를 원천 차단합니다.

```
$ git diff --cached --stat
 .gitignore                | 5 +++++
 2 files changed, 8 insertions(+)
```

02

브랜치와 병합



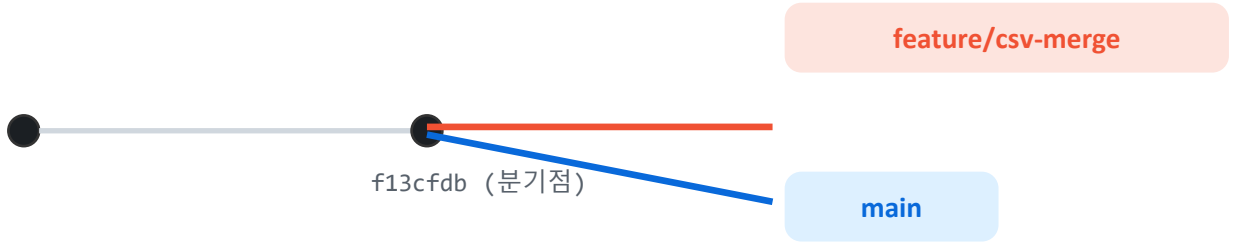
branch · checkout · merge — 독립적인 개발 흐름을 만들고 다시 합치기

git branch → git checkout — 가지치기와 이동

```
$ git branch feature/csv-merge
$ git branch
  feature/csv-merge
* main

$ git checkout feature/csv-merge
Switched to branch 'feature/csv-merge'
```

git branch <이름>은 브랜치를 **만들기만** 합니다(이동 X). 브랜치는 사실 "어떤 커밋을 가리키는 이름표"에 불과해서 생성이 매우 가볍고 빠릅니다.



각 브랜치에서 독립적으로 커밋하기

```
# feature/csv-merge 브랜치에서
(rdm_merge.py 작성)
$ git commit -m "CSV 병합"
[feature/csv-merge 072fb10]
```

```
# main 브랜치로 돌아와서
(README.md 수정)
$ git commit -m "README 추가"
[main 137d296]
```

git log --oneline --all --graph

```
* 137d296 README에 프로젝트 구조 섹션 추가 (main 브랜치)
| * 072fb10 CSV 병합 기능 추가 (feature/csv-merge 브랜치)
|/
* f13cfdb 데이터 생성기에 pandas 적용
* a1e0218 gitignore 추가 및 데이터 생성기 스켈레톤 추가
* 7d8eece 초기 커밋: README 추가
```

--graph 옵션으로 브랜치가 실제로 갈라진 모습을 시각적으로 볼 수 있습니다. f13cfdb에서 두 갈래로 나뉘어 각 브랜치가 독립적으로 발전했습니다.

git merge — 충돌 없는 병합

```
$ git merge feature/csv-merge -m "feature/csv-merge 병합"
Merge made by the 'ort' strategy.
 src/rdm_merge.py | 5 +++++
 1 file changed, 5 insertions(+)
```

```
*   5414211 feature/csv-merge 병합
|\
| * 072fb10 CSV 병합 기능 추가 (feature/csv-merge 브랜치)
* | 137d296 README에 프로젝트 구조 섹션 추가 (main 브랜치)
|/
* f13cfdb 데이터 생성기에 pandas 적용
  ...
```

서로 다른 파일을 건드렸기 때문에 Git이 자동으로 두 변경사항을 합쳐 새로운 **merge commit**(5414211)을 만들었습니다. 그래프에서 두 갈래가 다시 하나로 합쳐지는 모습(|\ → |/)을 확인할 수 있습니다.

충돌(Conflict) — Git 학습의 진짜 고비

같은 파일의 같은 부분을 두 브랜치에서 서로 다르게 수정하면, Git은 어느 쪽이 맞는지 판단할 수 없어 사람의 결정을 요구합니다.

```
$ git merge feature/analysis -m "feature/analysis 병합 시도"
Auto-merging src/rdm_analysis.py
CONFLICT (add/add): Merge conflict in src/rdm_analysis.py
Automatic merge failed; fix conflicts and then commit the result.
```

충돌난 파일을 열어보면:

```
def analyze(df):
<<<<<<< HEAD
    """결측치 분석 - main 버전"""
    return df.isnull().sum()
=======
    """기본 통계 분석 - feature/analysis 버전"""
    return df.describe()
>>>>>>> feature/analysis
```

충돌 마커의 의미

`<<<<<<< HEAD`	현재 내가 있는 브랜치(main)의 내용 시작
`=======`	두 브랜치 내용의 경계선
`>>>>>>> feature/analysis`	병합해오려는 브랜치의 내용 끝

충돌 해결 3단계

1. 직접 수정

마커를 포함해서 파일을
원하는 최종 상태로 수정

2. git add

"해결했다"고
Git에게 표시

3. git commit

병합을
완료

충돌 해결하고 병합 완료하기

```
# 수정 후 최종 내용
def analyze(df):
    """결측치 분석 + 기본 통계 분석 (두 기능 통합)"""
    return {
        "missing": df.isnull().sum(),
        "describe": df.describe()
    }
```

```
$ git add src/rdm_analysis.py
$ git status
All conflicts fixed but you are still merging.

$ git commit -m "merge: 두 버전의 분석 모듈 통합"
[main 5dd132c] merge: 두 버전의 분석 모듈 통합
```

Tip: 병합을 없었던 일로 하고 싶다면 **`git merge --abort`**로 즉시 되돌릴 수 있습니다.

03

되돌리기



restore · amend · reset — 실수를 안전하게 되돌리는 다양한 방법들

git restore — 워킹 디렉터리 되돌리기

커밋 전, 파일을 실수로 수정했을 때

```
$ echo "이것은 실수로 추가한 내용입니다" >> README.md
$ git restore README.md
→ 마지막 커밋 상태로 즉시 복구됨
```

add까지 했는데 취소하고 싶을 때 — git restore --staged

```
$ git add temp.txt
$ git restore --staged temp.txt
$ git status
Untracked files:
    temp.txt
```

git restore --staged는 Staging Area에서만 빼고, 파일 자체는 그대로 둡니다. "add를 취소"하는 명령으로 기억하세요.

git commit --amend — 마지막 커밋 수정

```
$ git commit -m "오타난 커밋 메세지ㅁㄴㅇ"
[main c2b943e] 오타난 커밋 메세지ㅁㄴㅇ

$ git commit --amend -m "임시 모듈 추가"
[main 6c3eff6] 임시 모듈 추가
```

c2b943e → 6c3eff6 커밋 해시 자체가 바뀐다

--amend는 새 커밋을 만드는 게 아니라 직전 커밋을 완전히 대체합니다. 메시지뿐 아니라 add로 추가한 파일까지 직전 커밋에 합쳐 넣을 수 있습니다.

주의: --amend는 커밋 해시를 바꾸기 때문에, 이미 다른 사람과 공유(push)한 커밋에 사용하면 협업자의 이력과 충돌이 날 수 있습니다. 아직 나만 보고 있는 커밋에서만 사용하는 것이 안전합니다.

git reset — soft / mixed / hard의 차이

```
$ git log --oneline -3
6c3eff6 임시 모듈 추가          ← 이 커밋을 취소하고 싶음
5dd132c merge: 두 버전의 분석 모듈 통합
5b21cae 분석 모듈 추가 (main, 다른 내용)
```

`--soft`: 커밋만 취소, 변경 내용은 Staging Area에 보존

```
$ git reset --soft HEAD~1
$ git status
Changes to be committed:
    new file:   src/temp_module.py
```

커밋 이력에서는 사라졌지만, 변경 내용은 여전히 staged 상태로 남아 있습니다. "커밋 메시지를 다시 쓰고 싶다" 같은 경우에 적합합니다.

git reset — mixed(기본값) / hard

--mixed (기본값)

```
$ git reset HEAD
$ git status
Untracked files:
    src/temp_module.py
```

Staging Area도 비웁니다. 파일 자체는 디스크에 남아 있습니다(이제 untracked).

--hard (위험!)

```
$ git reset --hard HEAD
$ ls src/
rdm_analysis.py  rdm_data_generator.py
rdm_merge.py   (temp_module.py 사라짐)
```

staged 상태였던 파일까지 깨끗하게 지웁니다. 되돌릴 수 없는 손실이 발생할 수 있습니다.

옵션	커밋 이력	Staging Area	Working Directory
`--soft`	되돌림	유지(staged)	유지
`--mixed (기본)`	되돌림	비움(unstaged)	유지
`--hard`	되돌림	비움	삭제 (주의!)

04

안전한 되돌리기 & 임시보관



revert · stash — 이력을 보존하며 되돌리고, 작업을 잠시 미뤄두기

git revert — 안전하게 되돌리기 (이력 보존)

```
$ git commit -m "설정 파일 추가 (버그 있음)"
[main e94e73a] 설정 파일 추가 (버그 있음)

$ git revert --no-edit HEAD
[main b7c50cc] Revert "설정 파일 추가 (버그 있음)"
 1 file changed, 1 deletion(-)
```

```
b7c50cc Revert "설정 파일 추가 (버그 있음)"   ← 새로 생긴 커밋
e94e73a 설정 파일 추가 (버그 있음)             ← 원래 커밋, 그대로 남음
5dd132c merge: 두 버전의 분석 모듈 통합
```

git reset
이력에서 커밋을 지워버림 (과거를 다시 씀)

git revert
원래 커밋은 두고, 상쇄하는 새 커밋 추가 (과거를 보존) — 공유된 이력에 항상 더 안전

git stash — 작업을 임시로 치워두기

작업 중

rdm_data_generator.py
수정 중 (미완성)

→

stash push

git stash push -m
"데이터 생성기 작업중"

→

긴급 작업

워킹 디렉터리 깨끗 →
핫픽스 커밋

→

stash pop

git stash pop
→ 작업 복원

```
$ git stash push -m "데이터 생성기 작업중"
Saved working directory and index state On main: 데이터 생성기 작업중

$ git status
nothing to commit, working tree clean    ← 안전하게 다른 작업 가능!
```

```
$ git stash pop
On branch main
Changes not staged for commit:
    modified:   src/rdm_data_generator.py
Dropped refs/stash@{0} (8482a1d3...)
```

05

이력 탐색



tag · show · blame · log graph — 기록 속에서 원하는 정보를 정확히 찾아내기

git tag — 릴리스 시점 표시

```
$ git tag -a v1.0.0 -m "GD RDM v1.0.0 - 초기 4대 기능 완성"
$ git tag
v1.0.0

$ git show v1.0.0 --stat | head -6
tag v1.0.0
Tagger: 최상길 <sangkil@example.com>
GD RDM v1.0.0 - 초기 4대 기능 완성
```

-a(annotated 태그)는 작성자·날짜·메시지가 포함된 정식 태그입니다. 단순히 **`git tag v1.0.0`**만 쓰면 메타데이터가 없는 가벼운(lightweight) 태그가 만들어집니다. 릴리스 표시용으로는 **-a**를 쓰는 것이 일반적입니다.

git show / git blame — 이력 들여다보기

git show — 특정 커밋 하나의 상세 변경 내용

```
$ git show f13cfdb --stat
commit f13cfdbfea89459778f6058ab47b227189bf7428
Author: 최상길 <sangkil@example.com>
 src/rdm_data_generator.py | 4 +++-
```

git blame — 파일의 각 줄을 누가, 언제 수정했는지 추적

```
$ git blame src/rdm_data_generator.py
f13cfdbf (최상길 2026-06-19 09:29:17) import pandas as pd
a1e0218a (최상길 2026-06-19 09:29:14) def generate_synthetic_data(rows=1000):
f13cfdbf (최상길 2026-06-19 09:29:17)     return pd.DataFrame(...)
```

각 줄 앞에 그 줄을 마지막으로 수정한 커밋·작성자·날짜가 표시됩니다. 협업 코드에서 "이 줄을 누가 왜 이렇게 썼는지" 추적할 때 필수적입니다.

정리

전체 이력을 한눈에 — 최종 그래프

```
$ git log --oneline --all --graph --decorate
* 0f5b426 (HEAD -> main, tag: v1.0.0) 데이터 생성기 작업 완료
* 64dc4fc 긴급 README 수정
* b7c50cc Revert "설정 파일 추가 (버그 있음)"
* e94e73a 설정 파일 추가 (버그 있음)
*   5dd132c merge: 두 버전의 분석 모듈 통합
|\
| * 239859f 분석 모듈 추가 (feature/analysis)
* | 5b21cae 분석 모듈 추가 (main, 다른 내용)
|/
*   5414211 feature/csv-merge 병합
|\
| * 072fb10 CSV 병합 기능 추가
* | 137d296 README에 프로젝트 구조 섹션 추가
|/
* f13cfdb 데이터 생성기에 pandas 적용
* a1e0218 gitignore 추가 및 데이터 생성기 스켈레톤 추가
* 7d8eece 초기 커밋: README 추가
```

이 한 줄의 그래프 안에 분기 · 병합 · 충돌 해결 · revert가 전부 기록되어 있습니다. Git의 이력은 지워지지 않고 누적되는 그래프입니다.

정리

전체 명령어 요약표

명령어	역할	영향 범위
`git init`	저장소 생성	-
`git add / git commit`	Staging → 저장소 기록	Staging / Repository
`git diff [--cached]`	변경사항 비교	-
`git branch / checkout`	브랜치 생성·이동	Working Directory
`git merge`	브랜치 병합	Repository
`git restore [--staged]`	되돌리기 (커밋 전)	Working / Staging
`git commit --amend`	직전 커밋 수정	Repository (해시 변경)
`git reset (soft/mixed/hard)`	커밋 되돌리기	단계별로 상이
`git revert`	안전하게 상쇄 커밋 추가	Repository (이력 보존)
`git stash`	작업 임시 보관/복원	Working Directory
`git tag / show / blame`	버전 표시 · 이력 조회	-

정리

실전 판단 가이드 — "이럴 땐 뭘 써야 하지?"

방금 수정한 파일을 망쳤다, 커밋은 안 했다	`git restore <파일>`
add까지 했는데 빼고 싶다	`git restore --staged <파일>`
커밋 메시지에 오타 (아직 공유 안 함)	`git commit --amend`
마지막 커밋을 취소하고 다시 만들고 싶다	`git reset --soft HEAD~1`
이미 push해서 남도 보는 커밋을 되돌려야 한다	`git revert`
작업 중인데 급하게 다른 브랜치로 가야 한다	`git stash`
두 기능을 독립적으로 개발하고 싶다	`git branch + checkout`
병합 중 같은 부분을 다르게 고쳤다(충돌)	`마커 수정 → add → commit`
이 버그를 언제, 누가 만들었는지 알고 싶다	`git blame`
릴리스 버전을 표시하고 싶다	`git tag -a`